

AI-based Village Pond Planning System

CSD Assignment 1 – Final Technical Report

RISHI KHARYA, Indian Institute of Technology Bhilai, India

Rural water security leans heavily on small rainwater-harvesting ponds, but deciding where to build one is usually a matter of judgement by eye, with no numbers to back it up. I built a web application that turns that judgement into a concrete calculation. The user draws a land area on a map, and for that area the backend pulls a real elevation model from OpenTopography, rebuilds the terrain, and works out how surface water would move across it using depression filling, D8 flow direction and flow accumulation. It then scores candidate pond sites with the topographic wetness and position indices, throws out anything too steep or sitting inside an existing stream, and traces the catchment that drains into each surviving site. Historical rainfall from Open-Meteo is combined with the catchment and the shape of the basin itself to estimate how much water a pond there could actually collect, and how deep it would need to be to hold a year's runoff. All of this — the suggested pond, its catchment, the terrain contours and the existing drainage — is drawn on the map and can be downloaded as an image, as GeoJSON or as JSON. The whole thing runs as a single lightweight process on the memory-limited servers we were given, finishes an analysis in around twelve seconds (and instantly once an area is cached), and is backed by 71 automated tests. None of the logic is tied to one specific place.

Additional Key Words and Phrases: Village Pond Planning, Geospatial Analysis, Rainwater Harvesting, Catchment Estimation, Web GIS, REST APIs

GitHub repository: <https://github.com/krish1809/village-pond>

Live application / API: <http://10.1.75.79:5228/> (API docs at /docs)

Demo video (YouTube): *[link to be added]*

1 INTRODUCTION

In most Indian villages, water availability is tied closely to how well the local rain is captured and stored. Ponds and tanks do a lot of that work: they hold monsoon runoff, let it soak in and recharge the groundwater, and carry a community through the dry months. Whether a pond actually helps, though, depends a great deal on where it is dug. A pond only makes sense where the surrounding land funnels runoff toward it and where the ground itself forms a hollow that can hold water. Put it on a slope, or in the middle of a stream, and it either drains away or simply passes the water on. Judging this well means reading the terrain, working out how much land drains to a point, and knowing how much rain that area gets.

The aim of this assignment was to build a web application that does this reasoning on its own. The user picks a region on a map, and the system suggests where ponds could go, shows the catchment feeding each one, and estimates the water each could collect, with everything drawn on the map. The rest of the report is organised as follows. Section 2 states the requirements and maps them to the code. Section 3 describes the architecture and how a request flows through the system. Section 4 covers the algorithms. Section 5 describes the implementation. Section 6 connects the work back to the course's system-design themes. Section 7 presents results and performance, and Section 8 discusses what the system does and does not do well.

Author's address: Rishi Kharya, Indian Institute of Technology Bhilai, Bhilai, India, kharyarishi1@gmail.com.

1.1 Motivation

Doing this by hand is slow and hard to get right. Someone has to walk the land and guess the drainage, there is no easy single place to combine the shape of the ground with years of rainfall data, and whatever comes out is qualitative — it gives no basis for choosing how deep a pond should be or how much it might store. A tool that automates the terrain and rainfall reasoning cuts a big search area down to two or three justified spots, each with real catchment and volume numbers, that an administrator can then go and check on site. It also makes the whole thing repeatable for any location, not just places an expert already happens to know.

1.2 Scope of the Project

For a selected area, the system estimates suitable pond locations, the catchment that drains to each, the pond's own footprint, the water volume it can realistically collect, and the depth needed to hold a year's runoff — and it draws all of that on the map and lets the user export it. What it does not do is also worth being clear about. It does not do the civil design of the pond (the embankment, spillway or soil work), it does not check from satellite imagery whether a pond already physically exists at a spot, and it does not model what happens below ground. The output is meant as decision support at the level of a survey shortlist, and its accuracy is limited by the resolution of the free elevation and rainfall data it relies on.

2 PROBLEM STATEMENT AND REQUIREMENTS

The central requirement is straightforward: let a user select a land area on a map and, for that area, produce and visualise the suggested pond location, the catchment area and the expected collectable water volume. Table 1 lists the functional requirements and points to the module that handles each one.

2.1 Non-Functional Requirements

Beyond the features, I set myself a few quality targets. A fresh analysis, including the live elevation download, should finish in well under a minute, and re-running the same area should feel instant. The service has to fit inside the provided containers, which are capped at 512 MB of RAM and are tight on storage. It should stay up on its own once started, without depending on someone being logged in. Because it leans on free external APIs, it must degrade gracefully when one of them is slow or down, rather than failing the whole request. It should work in a normal desktop browser with nothing to install, and the same build should run without changes on any of the four systems we were allocated.

3 SYSTEM ARCHITECTURE AND HIGH-LEVEL DESIGN

The system is organised into four logical layers with a clear separation between them: a presentation layer in the browser, an orchestration layer in the backend, an in-process analysis layer, and a set of external data services. There is intentionally no persistence layer (Section 5.3). What makes the deployment simple is that one FastAPI process plays two roles at once — it serves the compiled front-end as static files *and* answers the API calls — so the entire system is a single process behind a single URL. On the constrained containers this matters, because it keeps memory use, deployment steps and possible failure points to a minimum. Figure 1 shows the layers and the data flow between them.

Table 1. Functional requirements and where they are implemented

Requirement	Implemented in
Interactive map + satellite imagery	frontend/MapView (Leaflet; OSM base + Esri toggle)
Select land area on the map	frontend/App, MapView (view-select / two-click box)
Terrain / elevation acquisition	elevation_api.py (Open-Topography)
Contour visualization	contour_lines.py + MapView
Suitable pond-site identification	pond_locator.py
Catchment area estimation	catchment.py + geometry_utils.py
Historical rainfall query	rainfall_api.py (Open-Meteo)
Runoff / water-volume estimation	water_volume.py
Pond depth / storage recommendation	water_volume.py (storage curve)
Combined overlay / results view	frontend/MapView, ResultsPanel

3.1 What happens on one request

It is easiest to understand the design by following a single request end to end. When the user frames an area and presses *Analyse*, the front-end turns the map selection into a bounding box and sends it as JSON to `POST /analyzeArea`. The backend first validates the box (rejecting anything malformed, too large or too small), then checks its in-memory cache. On a miss it asks OpenTopography for a DEM covering exactly that box, requesting the plain-text ASCII grid format so no heavyweight geospatial libraries are needed to read it. That download is the slow part, so it is done asynchronously; the rest of the pipeline, which is pure NumPy, then runs quickly on the result.

From the raster the backend builds an elevation grid and a slope map, fills small artificial pits, computes flow direction and flow accumulation, and from the accumulation marks the existing stream network. It scores every cell for pond suitability, removes the steep cells and the stream cells, and picks a few well-separated candidate sites. For each candidate it traces the catchment upstream, works out the pond’s own flooded footprint, measures both in metres, and combines the catchment with the rainfall figure to estimate runoff, storage capacity, the collectable volume and a depth-volume curve. It also generates contour lines for display. Finally it packs everything — candidates, contours, drainage network, rainfall, a summary and a processing log — into one JSON response, which the front-end draws as layered overlays on the map. If any external call fails, the request does not: rainfall falls back to a documented default, and an unreachable DEM service returns a clear, actionable error instead of a stack trace.

In short, the pipeline runs as: validate the box and check the cache; fetch the DEM (async); build the elevation grid and slope; fill depressions, then D8 flow direction, then flow accumulation; rank candidate sites (excluding steep ground and channels); per site, trace the catchment and footprint and measure them in UTM; query rainfall and compute

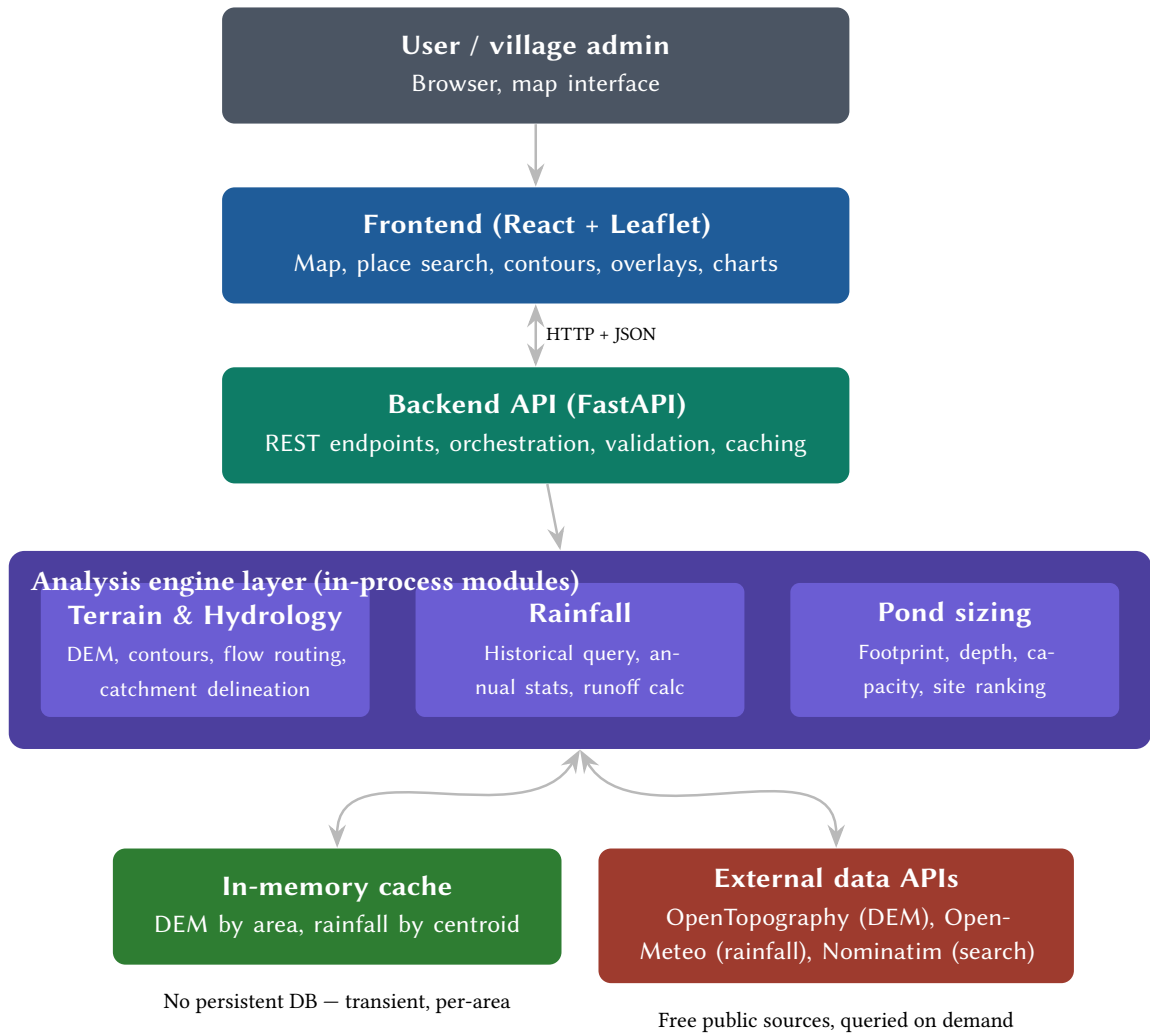


Fig. 1. High-level system architecture. The user's browser talks to the React front-end, which calls the FastAPI backend; the backend orchestrates the in-process analysis engine (terrain/hydrology, rainfall and pond-sizing modules), an in-memory cache and the external data APIs. There is no persistent database.

runoff, capacity and the storage curve; and finally assemble the JSON (with contours and the drainage network) that the front-end draws.

3.2 Component responsibilities

I kept each analysis module to a single, non-overlapping job so that a change in one place does not ripple into another. `elevation_api` fetches and parses the DEM; `terrain_grid` builds the grid and slope; `catchment` does the depression filling, flow direction, flow accumulation and watershed tracing; `pond_locator` scores and ranks the sites; `geometry_utils` measures areas and perimeters in metres; `rainfall_api` handles rainfall; `water_volume` does the

runoff, capacity and storage-curve maths; `contour_lines` produces the contours for display; and `visualization` renders the downloadable figure. The route handler itself stays thin and just calls these in order. On the front-end the split is similar: `App` holds the state and controls, `MapView` owns the Leaflet map and all the overlays, `ResultsPanel` shows the per-site numbers and charts, and `Search` does the place lookup.

3.3 Deployment view

On each server the built front-end is copied into the backend and served from there, and the process is started detached (with `nohup`) so it survives the session ending. Inside the container it listens on port 5000, which is forwarded out to port 5228, so the app is reached at the external address. Dependencies are installed into the environment already present on the machine rather than a fresh virtual environment, which keeps the storage footprint down. Because the whole system is one dependency-light process, the identical build runs on any of the four provided machines without modification.

3.4 Technology Stack

On the backend I used Python with FastAPI, which gives an async web layer and auto-generated API docs for free. NumPy and SciPy do the raster and hydrology work; Shapely and pyproj handle the polygon geometry and the UTM projection; `contourpy` extracts the contour lines; `httpx` makes the async external calls; and `matplotlib` (imported only when a figure is actually requested) renders the downloadable image. The front-end is React, bundled with Vite, using Leaflet for the map, with OpenStreetMap as the base layer and Esri World Imagery available as a satellite toggle. For data, elevation comes from the OpenTopography Global DEM API (SRTMGL1, 30 m), rainfall from the Open-Meteo historical archive, and place search from OpenStreetMap's Nominatim service.

Some of this differs from the stack suggested in the brief, and those changes were deliberate. The brief pointed at per-point elevation services and a PostGIS database. I moved to OpenTopography because it returns a whole DEM raster for the selected area in one request; the per-coordinate services I tried first kept hitting rate limits and became unusable during development. And I dropped the database entirely, because the analysis holds no state worth persisting and the containers are too storage-limited to justify one (Section 5.3). The satellite-based existing-pond detection sketched in the high-level design was left out of this phase.

4 METHODOLOGY

This section explains the actual algorithms behind the pipeline.

4.1 Terrain and Elevation Analysis

For the chosen bounding box the backend requests a DEM from OpenTopography (SRTMGL1, 30 m) in ESRI ASCII-grid form, which is plain text and can be read with NumPy alone — no native GDAL or rasterio build is needed, which matters on the lab containers. I orient the raster so that latitude increases with the row index, fill in any no-data cells, and, if the grid is larger than a set cap, downsample it by a fixed stride to keep memory in check. Slope at each cell comes from the Horn (1981) finite-difference gradient, $\text{Slope} = \arctan \sqrt{(dz/dx)^2 + (dz/dy)^2}$, after converting the cell spacing from degrees to metres. The elevation contours shown on the map are pulled straight from this grid using `contourpy`.

4.2 Catchment Area Delineation

A coarse DEM has small spurious pits that would trap water, so I first fill them with the Priority-Flood algorithm (Barnes et al., 2014), an $O(n \log n)$ method that processes cells outward from the boundary in order of elevation. Next, D8 flow direction (O’Callaghan & Mark, 1984) sends each cell to its steepest downhill neighbour, and flow accumulation — how many upstream cells drain through each one — is found by walking the flow graph in topological order. To get the catchment of a candidate site I trace this graph backwards from the site, adding a neighbour whenever its flow direction points at the current cell, until nothing upstream is left. The resulting set of cells is unioned into an exact polygon (I avoided a convex hull here, since testing showed it inflated the area badly) and reprojected into the local UTM zone, chosen automatically from the location, so the area and perimeter come out in metres. Two assumptions are worth stating: D8 sends all of a cell’s flow to one neighbour, and at 30 m the small valley-scale drainage on flat ground is only roughly captured, so catchments in flat areas are on the conservative side.

4.3 Rainfall Data Integration

Mean annual rainfall for the centre of the area comes from the Open-Meteo historical archive, using the daily precipitation totals and averaging over the last several complete years so that one unusually wet or dry year does not skew the figure; a month-by-month climatology is returned as well for the chart. Every call has a timeout, and if the archive cannot be reached the system falls back to a documented default annual rainfall and labels the source as such, so a volume estimate is always available.

4.4 Pond Siting and Water Volume

Two terrain measures decide where a pond makes sense. The Topographic Wetness Index, $TWI = \ln(a/\tan \beta)$, captures how strongly water gathers at a cell given its slope, and the Topographic Position Index, $TPI = z - \bar{z}_{\text{nbhd}}$, captures whether a cell sits below its surroundings. I combine them as $\text{Score} = 0.5 TWI_n + 0.5 (-TPI)_n$, then exclude anything steeper than 15° , any cell in the existing channel network (flow accumulation above the 97th percentile plus a buffer), and any candidate whose flooded footprint cannot be contained. For volume, the runoff the catchment delivers in a year is estimated with the Rational method, $V_{\text{runoff}} = C P A_{\text{catchment}}$ with $C = 0.3$ by default; the basin’s storage capacity is read straight off the DEM as $\sum (w - z_i) A_{\text{cell}}$; and the water that can actually be collected is the smaller of the two, $\min(V_{\text{runoff}}, V_{\text{capacity}})$. I also build a stage–area–volume curve and report the depth at which the stored volume first reaches a year’s runoff.

5 IMPLEMENTATION

5.1 Backend and API Design

The backend exposes a small REST surface (Table 2). Everything goes in and out as JSON, the shapes are defined with Pydantic models, and FastAPI turns those into interactive documentation at /docs on its own. There is no authentication — it is an open, read-only analysis tool for the demo. Failures from the third-party APIs are handled explicitly rather than left to bubble up: a bad or out-of-range area gives back 422, an unreachable elevation service gives 502 with a message the user can act on, a rainfall failure quietly uses the fallback, and any unexpected error in a stage returns 500 naming the stage that broke.

Table 2. REST API endpoints

Method & path	Purpose
POST /analyzeArea	Analyse a selected map region (bbox)
POST /analyzeArea?format=image	Same, returns a rendered PNG
POST /analyzeContour	Analyse an uploaded contour KML/KMZ (Phase 2)
GET /health	Service health check
GET /docs	Auto-generated API documentation

5.2 Frontend and Visualization

The front-end (Figure 2) is a single-page React and Leaflet app. The user can type a village or town name to jump the map there, frame a village-sized block, and select it either with a one-click “use current view” button or by clicking two opposite corners of a rectangle — I added the two-click option because click-and-drag was awkward on a laptop trackpad. Once the analysis returns, the map layers on the elevation contours, the existing drainage in blue, and for each ranked site the pond footprint as a filled shape labelled with its area, the catchment as a dashed outline, and a marker at the pond point. The side panel lists the catchment area, pond footprint, annual runoff, basin capacity, collectable volume and required depth for each site, with a small storage-curve chart and a monthly rainfall chart, and the results can be downloaded as a PNG, as GeoJSON or as the full JSON.

5.3 Database and Storage

There is no persistent database. Each analysis is computed on the spot from live data and returned; there is simply no cross-session state to keep. To avoid asking the external APIs for the same thing twice, the fetched DEMs (keyed by bounding box) and rainfall results (keyed by the centre point) are held in an in-memory cache, which is why re-analysing the same or a nearby area comes back instantly. This was a conscious choice: the containers are tight on storage and memory, a database would add weight and one more thing that can fail for no real benefit here, and staying stateless is what lets the same build run on any of the four machines without setup. If the user wants to keep a result, the GeoJSON and JSON downloads let them do that locally.

6 CSD THEMES AND TOPICS APPLIED IN THE PROJECT

Table 3 connects the course’s system-design themes to where each one actually shows up in this project. Where I did not implement a theme, I have said so plainly and given the reason, rather than claim it.

If I had to point to the three decisions that mattered most, they would be caching, resilience and the choice of a monolith. Caching mattered because a single analysis is dominated by the roughly twelve-second DEM download; keying the cache on the bounding box turns any repeat or nearby selection into an instant response, which is what actually makes exploring the map usable. Resilience was not really optional: early on, the free per-coordinate elevation service kept hitting rate limits and the other providers I tried were unreachable, so I moved to OpenTopography’s single-request raster and made every external call fail softly — rainfall drops to a default, and elevation returns a message the user can act on. And I went with a monolith instead of the microservice design in the original plan precisely

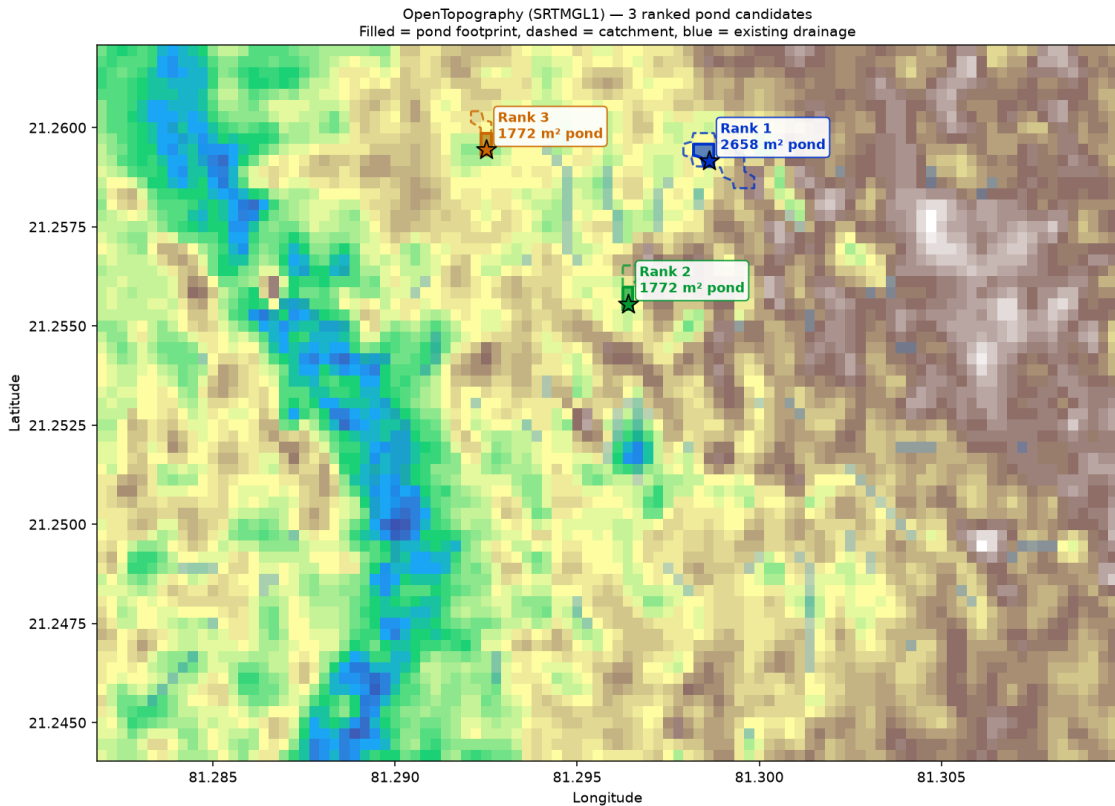


Fig. 2. Application results for the sample region: DEM terrain shading, the existing drainage network (blue), and three ranked pond sites (filled footprints) with their catchments (dashed), all kept off the watercourses.

because the targets are 512 MB containers; one small process that serves both the API and the web app is the design that fits the hardware I was given.

7 RESULTS AND EVALUATION

Figure 2 shows the system run on the sample region we were provided (near Raipur, Chhattisgarh). The 30 m DEM (elevations of 266–298 m) reproduces the landscape you would expect: the diagonal blue band is the main stream valley with its tributaries, and all three suggested ponds land on drainable ground clear of that network. Table 4 gives the numbers.

The numbers hang together the way they should: the catchment (the source of the water) is always at least as big as the pond footprint (where the water sits), and the collectable volume is the smaller of what the catchment can deliver and what the basin can hold. As a sanity check I compared this against Phase 2, which used a 1 m contour map of the same area, and the pipeline picks the same drainable zones and keeps clear of the river.

Table 3. Mapping of core CSD themes/topics to their use in this project

CSD Theme/Topic	Where used in the project	Justification / design rationale
API Design (REST)	POST /analyzeArea, /analyzeContour, /health	A clear JSON request/response contract via Pydantic, with docs generated automatically; a query flag (format=image) switches the representation without needing a separate route.
Caching	In-memory cache of DEM (by bbox) and rainfall (by centroid)	The external DEM download dominates the response time; caching turns repeat and nearby analyses into instant responses and removes duplicate external calls.
Concurrency / Asynchronous Processing	async route with httpx.AsyncClient for the DEM fetch	The request is I/O-bound on the external API, so an async fetch keeps the event loop free; the fast NumPy pipeline runs once the data is back.
Monolith vs. Microservices	One FastAPI process serving both the API and the static front-end	I chose a monolith over the microservice sketch in the HLD because the containers are tiny (512 MB) and one process keeps memory, deployment steps and failure points to a minimum.
Error Handling and Resilience	Rainfall fallback, DEM 502 with guidance, bbox validation, retry/backoff	The system depends on free external APIs, so it has to degrade gracefully instead of failing a whole analysis; every failure has a defined behaviour.
Algorithms and Complexity	Priority-Flood ($O(n \log n)$), D8 + flow accumulation ($O(n)$), BFS watershed	Using optimal/linear algorithms keeps the compute negligible (~ 0.1 s), so the total time is set by the network, not the CPU.
Geospatial / Projection	UTM projection (pyproj), exact cell-union polygons (Shapely)	Areas have to be measured in metres, not degrees; the exact union avoids the roughly 78% overestimate a convex hull gave in testing.
Resource-Constrained Design Testing Strategy	Lazy matplotlib import, grid cap + down-sampling, point caps 71 unit and integration tests (Section 7)	These keep the resident memory around 160 MB, comfortably under the 512 MB container limit. Each module is tested on its own plus end to end, which gives confidence the pipeline is right and that refactoring it is safe.
Containerization / Deployment	Runs in the provided Docker containers; scripted pull + restart on internal port 5000 (forwarded to 5228)	Reproducible and redeployed with one command; it uses the environment already on the machine instead of a duplicate virtual environment to save space.
Version Control	Git and GitHub, small incremental commits	The history records every change and fix and keeps the deployed servers in sync through git pull.
Load Balancing	Not implemented	A single instance handles the expected demo load within the container; an Nginx reverse proxy across instances would be the way to scale out.
Authentication / Authorization	Not implemented	The tool is an open, read-only analysis service for the demo and stores no user data.
Database Indexing	Not applicable	There is no persistent database (Section 5.3); state is transient and cached in memory.
CI/CD	Not implemented	Deployment is a scripted git pull and restart, and the tests are run by hand before deploying.

Table 4. Ranked pond candidates for the sample region ($C = 0.3$). Catchment area is always $\geq$ pond footprint, as expected.

Rank	Location (lat, lon)	Catch. (m ²)	Pond (m ²)	Collect. (m ³ /yr)	Depth (m)
1	21.2592, 81.2986	6 202	2 658	1 901	2.5
2	21.2556, 81.2964	2 658	1 772	877	1.0
3	21.2594, 81.2925	2 658	1 772	877	1.0

7.1 Performance

A fresh analysis of a new area takes about twelve seconds, and almost all of that — around 11.6 s — is the OpenTopography download; the entire CPU pipeline (flow routing, siting, catchment, geometry and volume) runs in roughly 0.1 to 1.5 s. Once an area is cached the response comes back in well under a second. The server sits at about 160 MB of resident memory, well below the 512 MB limit. The backend passes 71 automated tests covering the parser, terrain grid, flow and catchment, pond locator, geometry, elevation parsing, rainfall, water volume and the image-rendering path.

8 DISCUSSION AND LIMITATIONS

On the whole the pipeline works reliably and I can explain every step of it, and moving to OpenTopography’s single-request raster was what turned a flaky prototype into something dependable after the per-coordinate services had let me down. The real limits come from the free data and the scope I set. At 30 m the DEM cannot resolve small valley-scale drainage on flat ground, so catchments there stay modest and the D8 watershed can break into pieces; areas with genuine relief give much larger and cleaner catchments. The rainfall archive is sometimes unreachable, and when it is the system uses a documented fallback value and flags it, which makes those particular volume figures indicative rather than exact. The system also reasons only from terrain and rainfall: it does not know whether water already sits at a spot, and it does not do the civil design of the pond. None of these are bugs — they follow from the input data and the scope — but they are the reason the output is best treated as a shortlist to check on the ground rather than a final design.

9 AI TOOL USAGE DECLARATION

I used Claude while working on this project. It helped me draft first versions of some of the modules, talk through the terrain-analysis algorithms I was implementing, and track down a few bugs during testing — for example, a case where a pond’s footprint came out larger than its own catchment, which I fixed by clipping the footprint to the catchment, and the elevation service hitting rate limits, which led me to switch providers and add fallbacks. It also helped me fit this write-up into the report template. I went through everything it produced, changed what needed changing, and I can explain the algorithms and the design decisions in this report on my own.

A SOURCE CODE AND REPOSITORY

The full source code, the commit history and this report live at <https://github.com/krish1809/village-pond>, and the running application is at <http://10.1.75.79:5228/> (with the API docs at /docs). The repository is laid out as follows: backend/ holds the FastAPI service, with modules/ for the analysis code, api/routes/ for the endpoints, models/ for the Pydantic schemas and tests/ for the tests; frontend/ holds the React and Leaflet app under src/; scripts/ holds the build and deploy scripts; docs/ holds the design documents and this report; and the sample contour file sits at the project root.